

Kapitel 14

Strukturen

14.1 Vereinbarung von Strukturen

Wir haben gesehen, dass man Elemente mit gleichem Typ zu Feldern zusammenfassen kann. Häufig ist es aber auch wünschenswert, Objekte mit unterschiedlichem Typ zu logischen Einheiten zusammenfassen zu können: Will man etwa eine Studentendatei verwalten, benötigt man (u.a.) den Namen, die Matrikelnummer und die Adresse jedes Studenten. Aber auch für die Zusammenfassung von Objekten mit gleichem Typ bieten sich Felder in manchen Fällen nicht an: Einen Typ „komplexe Zahl“ kann man zwar durch

```
typedef double complex_t[2];
```

deklarieren; der Zugriff auf Real- und Imaginärteil durch Indizierung macht das Arbeiten mit komplexen Zahlen aber nicht gerade übersichtlich.

C bietet die Möglichkeit, unterschiedliche Objekte zu *Strukturen* zusammenzufassen.

Die Deklaration eines Strukturtyps beginnt mit dem Schlüsselwort **struct**. Ihm kann ein Name folgen, mit dem dann im weiteren Programm der Strukturtyp bezeichnet werden kann. Ihm folgt, in geschweifte Klammern eingeschlossen, die Aufzählung der Komponenten des Strukturtyps. Abgeschlossen wird die Deklaration durch ein Semikolon:

```
struct name {  
    typ komponente1;  
    typ komponente2;  
    ...  
    typ komponenteN;  
};
```

Als Beispiel wollen wir die bereits angesprochenen Studentendaten als Strukturtyp deklarieren:

```
struct student_s {  
    char vorname[30];  
    char nachname[30];  
    int matrikelnr;  
    int plz;  
    char wohnort[30];  
    char strasse[30];  
    int hausnr;  
};
```

Die Gültigkeit der Namen der Komponenten eines Strukturtyps ist auf den Strukturtyp beschränkt. Damit entstehen auch dann keine Namenskonflikte, wenn eine Komponente

denselben Namen wie eine Variable, ein Feld, eine Funktion oder auch eine Komponente eines anderen Strukturtyps besitzt.

In der beschriebenen Form handelt es sich um eine reine Deklaration, d.h. es wird noch keine Variable mit dem Typ definiert und entsprechend kein Speicherplatz bereitgestellt. Variablen mit einem Strukturtyp, kurz: Strukturen, kann man auf verschiedene Weisen definieren.

Zunächst kann man die Namen direkt zwischen die schließende geschweifte Klammer und das nachfolgende Semikolon der Typdeklaration setzen:

```
struct student_s {
    char vorname[30];
    /* ... wie zuvor */
} student1, student2;
```

Will man im Rest des Programms keinen Bezug mehr auf den Strukturtyp nehmen, kann man den Namen, hier also `student_s`, auch weglassen. Wenn der Name angegeben ist, kann man Strukturen auch so definieren:

```
struct student_s student1, student2;
```

Für das Aufschreiben und auch die Lesbarkeit eines Programms ist es in der Regel günstiger, Strukturtypen mit `typedef` Namen zu geben:

```
typedef struct student_s {
    char vorname[30];
    /* ... wie zuvor */
} student_t;
```

Jetzt kann man Strukturen etwa durch

```
struct student_s student1, student2;
student_t student3, student4;
```

definieren. Bei der Deklaration kann man den Strukturnamen `student_s` weglassen. Die erste Variante der Variablenvereinbarung ist dann jedoch nicht mehr möglich. Das ist jedoch kein Nachteil, da die zweite Variante ohnehin kürzer ist. Eine Kombination von `typedef`-Deklaration und Strukturdefinition ist *nicht* möglich.

Strukturnamen wurden bisher mit einem nachgestellten `_s` versehen. Das ist nicht zwingend erforderlich, verdeutlicht jedoch, dass es sich um einen Strukturnamen handelt. Generell haben Strukturnamen jedoch einen eigenen Namensraum, da sie ja nur zusammen mit `struct` verwendet werden können.

14.2 Operationen mit Strukturen

Operationen mit Strukturen als Ganzem sind nur sehr eingeschränkt möglich:

- Wertzuweisungen zwischen Strukturen sind möglich. Dabei werden die Werte aller Komponenten übertragen. Offensichtliche Voraussetzung ist die Identität der Strukturtypen.
- Funktionen können Strukturen als Parameter besitzen. Die Abarbeitung der Aufrufe erfolgt wie bei (einfachen) Variablen: Im Zuge des Aufrufs wird der Wert des Arguments in eine lokale Struktur der Funktion kopiert. Zumindest bei umfangreicheren Strukturen sollte man sich also genau überlegen, ob die Übergabe eines Zeigers auf die Struktur nicht zweckmäßiger ist.

- Funktionen können Strukturen als Funktionswerte zurückliefern. Wie bei den Parametern sollte man sich aber genau überlegen, ob das zweckmäßig ist. Auch hier kann ein Zeiger-Parameter die bessere Lösung sein.
- Mit dem Adressoperator & kann man die Adresse einer Struktur ermitteln.
- Bei der Definition einer Struktur kann man ihren Anfangswert festlegen, in der gleichen Form wie bei Feldern:

```
student_t student = {
    "Klaus", "Meier",
    12345,
    37073, "Goettingen",
    "Einbahnstrasse", 12
};
```

Mit den Komponenten von Strukturen kann man dagegen „ganz normal“ arbeiten. Allerdings reicht die Angabe des Namens einer Komponente offensichtlich nicht aus, da alle Strukturen mit gleichem Typ ja gleichnamige Komponenten besitzen. Erforderlich ist, ähnlich wie bei Zeigern, eine Dereferenzierung: Angegeben werden der Name der Struktur *und* der Name der Komponente; zwischen beide wird der *Punktoperator* (.) gesetzt:

```
student_t student;
...
student.plz = 1000;
```

Als Beispiel schreiben wir ein Programm, das den Datensatz eines Studenten liest und wieder schreibt:

```
#include<stdio.h>

/**  Strukturdeklaration  *****/
typedef struct {
    char vorname[30];
    char nachname[30];
    int  matrikelnr;
    int  plz;
    char wohnort[30];
    char strasse[30];
    int  hausnr;
} student_t;

/**  Prototypen  *****/
student_t gelesener_satz (void);
void satz_schreiben (student_t s);

/*= Hauptprogramm =====*/
int main(void) {
    student_t satz;
    printf("Datensatz eingeben:\n");
    satz = gelesener_satz ();
    printf("Eingegebener Datensatz:\n");
    satz_schreiben (satz);
    return 0;
}

/*= Eingabefunktion =====*/
student_t gelesener_satz(void) {
```

```

    student_t s;

    printf("Vorname: ");
    scanf("%s", s.vorname);
    printf("Nachname: ");
    scanf("%s", s.nachname);
    printf("Matrikelnummer: ");
    scanf("%d", &s.matrikelnr);
    printf("Postleitzahl: ");
    scanf("%d", &s.plz);
    printf("Wohnort: ");
    scanf("%s", s.wohnort);
    printf("Strasse: ");
    scanf("%s", s.strasse);
    printf("Hausnummer: ");
    scanf("%d", &s.hausnr);
    return s;
}

/*= Ausgabefunktion =====*/
void satz_schreiben(student_t s) {
    printf("Vorname: %s\n", s.vorname);
    printf("Nachname: %s\n", s.nachname);
    printf("Matrikelnummer: %d\n", s.matrikelnr);
    printf("Postleitzahl: %d\n", s.plz);
    printf("Wohnort: %s\n", s.wohnort);
    printf("Strasse: %s\n", s.strasse);
    printf("Hausnummer: %d\n", s.hausnr);
}

```

Quelltext 14.1: Arbeiten mit Strukturen

14.3 Schachtelung strukturierter Typen

Jede Strukturkomponente darf einen beliebigen Typ besitzen. Wir haben das zum Beispiel schon genutzt, indem wir Felder als Strukturkomponenten deklariert haben. Entsprechend kann man Felder definieren, deren Komponenten Strukturen sind. Außerdem darf jede Strukturkomponente ihrerseits eine Struktur sein.

Zum Beispiel können wir vereinbaren

```

typedef struct {
    char vorname[30];
    char nachname[30];
} name_t;

typedef struct {
    int plz;
    char wohnort[30];
    char strasse[30];
    int hausnr;
} adresse_t;

typedef struct {
    name_t name;
    int matrikelnr;
    adresse_t adresse;
}

```

```

} student_t;

student_t student, studenten[50];

```

Für die Zugriffe auf einzelne Komponenten muss man dann die „Pfade“ nachverfolgen, zum Beispiel

```

student.adresse.hausnr = 17;
if (studenten[17].name.vorname[0] == 'K')
    ...

```

Die Abarbeitung solcher Pfade erfolgt, wie es nahe liegt, von links nach rechts.

14.4 Zeiger auf Strukturen

Es wurde bereits angesprochen, dass man sich sehr genau überlegen sollte, ob man umfangreiche Strukturen als Parameter von Funktionen oder als Funktionstypen vorsieht, weil beides das Kopieren ganzer Strukturen impliziert. In der Regel wird man deshalb im Zusammenhang mit Funktionen Zeiger auf Strukturen verwenden. Es ist übrigens nicht so recht nachvollziehbar, warum C nicht grundsätzlich Strukturen wie Felder behandelt, also Strukturnamen als Zeiger auf den Anfang der Struktur betrachtet.

Für Zeiger auf Strukturen gelten dieselben Regeln wie für Zeiger auf einfache Variablen:

```

student_t student, *p = &student;

```

Beim Zugriff auf die Komponenten muss man berücksichtigen, dass der Punktoperator höhere Priorität besitzt als der Dereferenzierungsoperator (*). Wir *müssen* also

```

(*strukturzeiger).strukturkomponente

```

schreiben. Abkürzend und suggestiver ist die äquivalente Schreibweise mit dem Pfeil-Operator (->).

```

strukturzeiger->strukturkomponente

```

Sehen wir uns als Beispiel noch einmal die Ein-/Ausgabe an. Diesmal sind die Strukturen verschachtelt und es werden den Funktionen Zeiger auf die Strukturen übergeben, um das automatische Kopieren der ganzen Strukturen zu vermeiden.

```

#include<stdio.h>

/**  Strukturdeklarationen  *****/
typedef struct {
    char vorname[30];
    char nachname[30];
} name_t;

typedef struct {
    char strasse[30];
    int  hausnr;
    int  plz;
    char wohnort[30];
} adresse_t;

typedef struct {
    name_t    name;
    int       matrikelnr;
}

```

```

    adresse_t adresse;
} student_t;

/** Prototypen *****/
void satz_lesen(student_t *const s);
void satz_schreiben(const student_t *const s);

/*= Hauptprogramm =====*/
int main(void) {
    student_t satz;
    printf("Datensatz eingeben:\n");
    satz_lesen (&satz);
    printf("Eingegebener Datensatz:\n");
    satz_schreiben (&satz);
    return 0;
}

/*= Eingabefunktion =====*/
void name_lesen(name_t *const n) {
    printf("Vorname: ");
    scanf("%s", n->vorname);
    printf("Nachname: ");
    scanf("%s", n->nachname);
}

void adresse_lesen(adresse_t *const a) {
    printf("Postleitzahl: ");
    scanf("%d", &a->plz);
    printf("Wohnort: ");
    scanf("%s", a->wohnort);
    printf("Strasse: ");
    scanf("%s", a->strasse);
    printf("Hausnummer: ");
    scanf("%d", &a->hausnr);
}

void satz_lesen (student_t *const s) {
    name_lesen(&s->name);

    printf("Matrikelnummer: ");
    scanf("%d", &s->matrikelnr);

    adresse_lesen(&s->adresse);
}

/*= Ausgabefunktion =====*/
void satz_schreiben (const student_t *const s) {
    printf("Vorname: %s\n",      s->name.vorname);
    printf("Nachname: %s\n",    s->name.nachname);
    printf("Matrikelnummer: %d\n", s->matrikelnr);
    printf("Postleitzahl: %d\n",  s->adresse.plz);
    printf("Wohnort: %s\n",      s->adresse.wohnort);
    printf("Strasse: %s\n",      s->adresse.strasse);
    printf("Hausnummer: %d\n",   s->adresse.hausnr);
}

```

Quelltext 14.2: Arbeiten mit verschachtelten Strukturen und Zeigern auf Strukturen

14.5 Strukturen auf dem Heap

Für Zeiger auf Strukturen gelten dieselben Regeln wie für Zeiger auf einfache Variablen. Insbesondere heißt das auch: Definiert man eine Zeigervariable, so wird nur der Speicher für die Aufnahme des Zeigers bereitgestellt. Durch Zuweisung eines Zeigers auf eine statische oder automatische Struktur kann man solch einer Zeigervariablen einen Wert geben. Aber auch Bereitstellung von Speicher auf dem Heap ist möglich.

```
student_t *p;
...
p = (student_t *)malloc(sizeof (student_t));
```

Hier ist der Operator `sizeof` unentbehrlich! Wieviel Speicher eine Struktur benötigt, kann nämlich von Rechner zu Rechner sehr unterschiedlich sein, selbst wenn die einzelnen Komponenten auf den Rechnern jeweils gleichviel Speicher benötigen. Dahinter steht eine Vorschrift des Standards: Die Komponenten einer Struktur müssen im Speicher des Rechners zwar dieselbe Reihenfolge besitzen wie in der Deklaration – sie müssen aber nicht lückenlos aufeinanderfolgen! Entsprechend addiert der Operator `sizeof` nicht einfach den Speicherbedarf der Komponenten, sondern berücksichtigt auch eventuelle Lücken. Als Grundregel sollte man sich einfach merken: Bei jedem Aufruf von `malloc` steckt im Argument der Operator `sizeof`!

14.6 Datum und Uhrzeit

Mit der Kenntnis von Strukturen kann nun auch die Headerdatei `time.h` besprochen werden. Aktuelles Datum und Uhrzeit kann man sich von der Funktion

```
time_t time(time_t *zeit);
```

liefern lassen – in einer implementationsspezifischen Darstellung. Dabei:

- Der Funktionswert `(time_t)(-1)` (*Typecast*) markiert, dass Datum und Uhrzeit nicht verfügbar sind.
- Wenn `zeit` der Nullzeiger ist, wird nur der Funktionswert geliefert; sonst wird das Resultat zusätzlich auch in die angegebene Variable geschrieben.

Die Headerdatei bietet aber auch die Möglichkeit, daraus eine allgemein verständliche Darstellung erzeugen zu lassen. In ihr ist der Strukturtyp

```
struct tm
```

definiert. Er muss zumindest die folgenden Komponenten besitzen:

<code>int tm_sec</code>	Sekunde in der Minute, 0...61
<code>int tm_min</code>	Minute in der Stunde, 0...59
<code>int tm_hour</code>	Stunde seit Mitternacht, 0...23
<code>int tm_mday</code>	Tag im Monat, 1...31
<code>int tm_mon</code>	Monat im Jahr, 0...11
<code>int tm_year</code>	Jahr seit 1900
<code>int tm_wday</code>	Tag in der Woche (ab Sonntag), 0...6
<code>int tm_yday</code>	Tag seit 1. Januar, 0...365
<code>int tm_isdst</code>	Sommerzeit-Marke (daylight saving time) (<code>< 0</code> : unbekannt)

wobei die Reihenfolge der Komponenten durch den Standard nicht festgelegt ist. Die Umwandlung eines Wertes mit dem Typ `time_t` in den Typ `tm` nehmen die Funktionen

```
struct tm *gmtime(const time_t *zeit);
struct tm *localtime(const time_t *zeit);
```

vor – in Greenwich-Standardzeit bzw. in die lokale Zeit. Daneben gibt es auch die Umkehrung: Die Funktion

```
time_t mktime(struct tm *zeitpunkt);
```

wandelt den Wert der Struktur in die interne Darstellung um. Sie unterstellt, dass es sich um eine lokale Zeit handelt.

14.7 Verkettete Listen

Strukturen und Zeiger auf Strukturen bilden die Grundlage einer wichtigen nicht elementaren Datenstruktur, nämlich der *verketteten Liste*. Wegen der Bedeutung der verketteten Listen soll ein kurzer Überblick geben werden.

Anschaulich kann man verkettete Listen mit „Schnitzeljagd“ vergleichen:

- Man hat einen Verweis, wo man beginnen muss (Zeigervariable).
- Am Start findet man einen Zettel mit einer Aufgabe und einen neuen Verweis (Struktur mit Daten- und Zeigerkomponente).
- Folgt man den Verweisen, kommt man irgendwann zum Ende (Zeigerkomponente mit Nullzeiger).

Abbildung 14.1 verdeutlicht das.

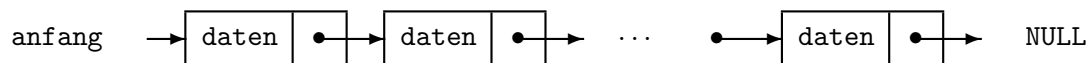


Abbildung 14.1: Verkettete Liste

Im Programm kann man das durch Deklarationen wie

```
struct liste {
    DATENTYP daten;
    struct liste *nachfolger;
};
```

nachvollziehen. Interessant ist, dass hier scheinbar eine „rekursive Deklaration“ vorliegt. Tatsächlich ist die zweite Komponente aber ein Zeigertyp und steht nur für eine Adresse, nicht für den Strukturtyp, der gerade deklariert wird.

Standardoperationen für verkettete Listen sind

- Einfügen eines neuen Eintrags.
- Suchen eines Eintrags mit bestimmtem Wert.
- Entfernen eines Eintrags.

Typischerweise werden verkettete Listen auf dem Heap angelegt. Der Fantasie sind beim Aufbau verketteter Strukturen keine Grenzen gesetzt:

- In einer verketteten Liste kann jeder Eintrag Anfang einer verketteten Liste sein.
- Mehrere Zeigerkomponenten in jeder Struktur erlauben den Aufbau von mehrfach verketteten Listen oder auch von *Bäumen*.

14.8 Verbunde

Formal den Strukturen sehr ähnlich sind die *Verbunde*: Statt `struct` schreibt man `union`. Inhaltlich bestehen jedoch ganz gravierende Unterschiede:

- Bei Strukturen belegen aufeinanderfolgende Komponenten aufeinanderfolgende Plätze im Speicher, wenn auch vielleicht nicht lückenlos, wie wir gesehen haben.
- Bei einem Verbund beginnen alle Komponenten an derselben Speicheradresse. Der Verbund belegt insgesamt so viel Speicher wie seine längste Komponente. Dadurch wird erreicht, dass der gleiche Speicherplatz zu unterschiedlichen Zeitpunkten in unterschiedlicher Weise genutzt werden kann, z.B. je nach Situation zur Speicherung eines `float`-Wertes oder (wenn dieser gerade nicht benötigt wird) zur Speicherung eines `int`-Wertes.

In der Praxis kommen Verbunde recht selten vor. Ein kurzes Beispiel soll zeigen, dass ihre Verwendung extrem leicht zu Fehlern führt. Wir hatten gesehen, wie man sich selber mit Castoperatoren „austricksen“ kann, etwa indem man

```
int i, *ip = &i;
float *fp;
...
fp = (float *) ip;
*fp = 6.5f;
printf("%d\n", i);
```

schreibt. Mit einem Verbund kann man dasselbe einfacher haben:

```
union {
    int i;
    float f;
} quatsch;
...
quatsch.f = 6.5f;
printf("%d\n", quatsch.i);
```

